

Run as IDS

```
sudo snort -A console -q -c /etc/snort/snort.conf -i eth0
```

Reality check: Snort requires tuning. The default rules generate false positives. You'll spend time adjusting thresholds and disabling noisy rules. This is normal. Security monitoring is inherently noisy—your job is separating signal from static.

Suricata: Snort's faster sibling

This IDS/IPS engine does everything Snort does, but is faster and has more features.

Why choose it over Snort?

- *Multi-threaded:* Uses all your CPU cores (Snort is single-threaded).
- *File extraction:* Automatically pulls files from network traffic for analysis.
- *Lua scripting:* Writes complex detection logic.
- *Native TLS inspection:* Decrypt and analyse HTTPS (with proper certificates).

When to use it:

- High-bandwidth networks where Snort chokes
- When you need file extraction and reconstruction
- Modern deployments requiring multi-core performance
- When you want IDS and IPS in one tool

First steps:

Ubuntu/Debian

```
sudo apt install suricata
```

Download emerging threats rules

```
sudo suricata-update
```

Basic test run

```
sudo suricata -c /etc/suricata/suricata.yaml -i eth0
```

Check results

```
cat /var/log/suricata/fast.log
```

The IPS mode warning: Suricata can block traffic, not just detect it. Enabling IPS is powerful but dangerous. A bad rule can drop legitimate traffic. Start with IDS mode. Get comfortable. Then consider IPS with careful testing.

Trivy: The container scanner that just works

This vulnerability scanner for containers and other artifacts finds CVEs in OS packages and application dependencies.

Containers are built from layers—OS base images, package installations, application code. Each layer can carry vulnerabilities. Trivy checks them all without requiring a running container.

What it scans:

- Container images (Docker, containerd, Podman)
- Filesystems and rootfs
- Git repositories
- Kubernetes clusters
- Infrastructure as Code (Terraform, CloudFormation)
- Software Bill of Materials (SBOMs)

Getting started:

Install

```
curl -sL https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install.sh | sh -s -- -b /usr/local/bin
```

Scan a public image

```
trivy image python:3.9
```

Scan local project

```
trivy filesystem /path/to/project
```

Output as JSON for automation

```
trivy image --format json --output report.json nginx:latest
```

Scan Kubernetes cluster

```
trivy kubernetes --report=summary cluster
```

CI/CD integration: This is where Trivy shines. Add it to your pipeline and fail builds with critical vulnerabilities:
Yaml

GitHub Actions example

```
- name: Run Trivy vulnerability scanner
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: 'myapp:${{ github.sha }}'
    format: 'table'
    exit-code: '1'
    ignore-unfixed: true
    severity: 'CRITICAL,HIGH'
```

Critical vs high vs low: Not all CVEs demand immediate action. A vulnerability in a package you don't use, in a container with no network access, is different from a remote code execution in your public-facing API. Trivy gives you data; you need judgment.

Other tools

These tools deserve a mention:

OpenVAS/Greenbone: Vulnerability scanning for entire networks. Think Nessus, but free. Scans hosts, finds open ports, identifies outdated software, checks configurations.